The assert line is doing the work: an anchor-leak ablation of EvaLoop

Shreyas Visweshwaran · CSC 591/791 Trustworthy AI for Software Engineering · Assignment 2 Fork:
https://github.com/FirefoxSRV/EvaLooop (branch `anchor-ablation`)

1. Extension and research question

EvaLoop measures robustness by feeding a model its own output: generate code from a description, test it, summarize the code, and let that summary become the next description. Three measurements on the provided data package suggested the loop is doing less work than it appears to. Across all 107 leaderboard models, Spearman(pass@1, ASL) = **0.945** (Pearson $R^2 = 0.81$), so ASL is nearly rank-equivalent with one-shot accuracy. About **91%** of loops that pass cycle 1 never fail within ten cycles, and the median model has only **26** mid-loop failures out of 378 tasks.

One string in the loop never degrades. `CodeSummarizationTask.postprocess` appends the **final assert line of the original dataset prompt** to every summary, forever. It is there for a real reason — a summary does not fix the function's name, so the tests could not call the generated code — but it carries more than a name: one concrete input together with its **ground-truth expected output**, re-injected from outside the loop on every cycle, in a framework whose premise is that the model's own output is the only source of perturbation. Models exploit it: **33.4%** of all 165,630 recorded cycles contain an echoed module-level assertion (99.5% for gpt-5, 0% for gemini-2.0-flash).

> ***RQ.*** *How much of EvaLoop's measured robustness is the model sustaining its own specification, and how much is a re-injected ground-truth anchor holding the task still?*

I expected removing the oracle to lower ASL and raise mid-loop failures (H1), and — if the mechanism is lost test information rather than wandering specifications — to produce failures preceded by *less* semantic drift (H3). Hypotheses were fixed in `PREREGISTRATION.md` before any run.

2. The framework and my changes

The pipeline. `EvaLoopEvaluator` loads the 378 MBPP+ tasks and seeds each with the benchmark's docstring prompt. For each cycle, `BatchEvaluationCycle` runs the configured tasks in sequence through one interface — `create_prompt → model → extract_output → postprocess`. The generation task renders `"Generate Python code for the following task: …"`, and extracts the first triple-backtick block from the reply. That code is executed against MBPP+'s base and extended EvalPlus suites via `untrusted_check`; a cycle passes only if both pass. If it passed, the summarization task renders a one-sentence summarization prompt over the code, and its `postprocess` wraps the reply as a docstring with the assert line re-appended — which becomes the next cycle's description. A task that fails stops; survivors continue to a maximum of ten cycles. `successful_cycles` is cycles completed before the first failure, and `evaloop/analysis/metrics.py` turns the per-task vector into $\text{ASL} = \Sigma_i\ n_i \cdot i^2/(T \cdot M)$, where the i^2 weighting pays compound credit for sustained cycles.

What I changed. The anchor was the only thing I needed to vary, and the `dev` branch already routes it through `postprocess`, so the extension is almost entirely new files: `evaloop/anchors/` (a registry mirroring `tasks.factory`, plus three strategies), two new task types, a Gemini backend, a disk-backed call cache, a sandboxed batch tester, and an evaluator subclass. **Five existing source files changed, +43/−7 lines.** The one substantive edit is in `evaluation/cycle.py`: upstream's `postprocess` context carried only the cycle-start input, and a model-derived anchor needs the code that was *just produced*, so the context now also carries `task_input` (both the batch and sequential paths; existing tasks ignore it). The rest are lazy-import moves in `models/factory.py` and `core/evaluator.py` so the generation stage needs neither torch nor evalplus, plus two registration lines and a docstring. I reused `analysis/metrics.py` and `analysis/similarity.py` rather than reimplementing the metrics or the judge.

Arm	Anchor	Carries
A0 `assert`	`assert find_length("11000010001") == 6`	name, arity, inputs, **expected output** — upstream unchanged
A1 `masked_assert`	`assert find_length('11000010001') == ...`	name, arity, inputs; oracle masked by AST
A2 `signature`	`def find_length(s):`	name and parameter names, parsed from the model's own code each cycle

A1−A0 isolates the oracle; A2−A1 isolates the remaining ground-truth-derived argument values. A2 is the only arm where nothing but model output crosses the boundary. **Signature stability** — the problem the assignment requires any Option B design to solve — is handled by parsing the code that just *passed* and emitting the `def` line for the dataset's `entry_point`; since that code passed the tests, a callable function with the right name and arity exists by construction. Because cycle 1 is byte-identical across arms and the call cache is shared per model, **pass@1 is equal across arms by construction**, so ΔASL is attributable to the anchor alone.

One confound had to be controlled. A1's anchor ends `== ...`, so a model that echoes it raises `AssertionError` and fails for a formatting reason; and an echoed assertion is part of the code summarized next, which could restate the oracle in prose. Module-level `assert` statements are therefore stripped from generated code before testing *and* before summarization, in **every** arm including the baseline.

Validation before any claim. With `anchor: assert` the new task reproduces **all 165,630** recorded cycle transitions in the data package byte-for-byte, so the baseline arm is provably the published loop. `analysis/metrics.py` reproduces the leaderboard for **56/60** models (the other four are the multi-run models its README documents). Re-executing 120 stored solutions through my sandbox reproduces the recorded verdict for **117/120 (97.5%)**, and the assert-stripping rule changed **0 of 40** verdicts on samples that contained a module-level assertion — so the rule is immaterial to outcomes, as the design requires. The framework's own `ResultAnalyzer`, run on my exported results, returns my ASL to ten decimal places. 209 tests pass.

3. Experimental setup

Two models, `gemma-4-26b-a4b-it` and `gemma-4-31b-it`, via the Gemini API. The Gemini 2.5 series — the only models with baselines in the data package — now returns `404 … no longer available to new users`, so no evaluated model has a published baseline and **the A0 arm is the baseline**, which is what Option B requires anyway. 150 MBPP+ tasks, drawn by `scripts/sample_tasks.py` with seed 20251013: five equal quantile strata over each task's mean sustained cycles across the 60 package models, 30 per stratum, from the 362 tasks whose assert states a comparison (16 assert bare truthiness and admit no masked anchor). The subset reproduces the 60-model ASL ranking at Spearman **ρ = 0.937**. Greedy (`temperature=0`, `top_p=1`), `max_output_tokens=1024`, `seed=20251013`, thinking pinned to `minimal`, max 10 cycles, one run per configuration.

Untrusted code is executed only inside a container with `--network none`, a non-root user, dropped capabilities, a read-only repo mount and memory/CPU/pid caps; the test image holds no model SDK and no API key. The container is required rather than advisory: EvalPlus's `reliability_guard` sets `RLIMIT_AS`, which macOS rejects.

Cost. All runs were on the free tier: **$0.00 billed**. Measured usage for one model's full three-arm ablation: **760 API calls**, 122,466 tokens (72,725 in / 49,741 out), with 5,818 calls served from cache — i.e. **88.4% of all calls were byte-identical repeats**. That is ∼5 calls and ∼816 tokens per task-arm. No LLM was used as a judge or classifier for the results reported here; `scripts/judge_similarity.py` implements the boundary-similarity judge, reusing the framework's byte-identical prompt from the data package, and records in every run file that absolute scores would not be comparable to the published `gpt-4-turbo-2024-04-09` ones.

4. Results and failure analysis

arm	pass@1	ASL	ASL_bias	mid-loop failures	survived all 10	ΔASL [95% CI]	Wilcoxon
A0 `assert`	0.747	7.207	9.652	4	108	—	—
A1 `masked_assert`	0.747	6.943	9.298	8	104	−0.264 [−0.59, +0.00]	p = 0.066
A2 `signature`	0.747	6.681	8.947	12	100	**−0.526 [−0.92, −0.13]**	**p = 0.020**

(`gemma-4-26b-a4b-it`, 150 tasks; paired bootstrap, 10k task resamples; two-sided Wilcoxon signed-rank on per-task sustained cycles.)

ASL falls monotonically and mid-loop failures triple. The A0→A2 difference is significant; A1 alone is borderline, so most of the effect needs the concrete argument values gone as well as the expected output. **H1 is supported in direction but the magnitude is modest** — well below the >1.0 loop I pre-registered, and that matters for interpretation: the leak is load-bearing for a *minority* of tasks, not for robustness in general.

Two structural findings explain why, and neither is visible from ASL alone.

The sustained-cycle distribution is almost perfectly bimodal. Under A0, 38 tasks fail at cycle 1, 4 fail at cycles 2–3, and 108 survive all ten; **nothing fails between cycles 4 and 9**. The entire ablation effect lands on "sustained exactly one cycle" (2 → 6 → 9). The oracle matters precisely at the first handoff from the model's own description.

The loop reaches a fixed point almost immediately. 108 of the 110 tasks that get past cycle 1 reach a *repeated* (code, summary) state, at median **cycle 2** under A0 and cycle 3 under A2 — the model regenerates identical code, which yields an identical summary, indefinitely. This is what the 88.4% cache-hit rate is measuring. So cycles 4–10 re-derive a fixed point while ASL's i^2 weighting pays quadratic credit for them, and weakening the anchor measurably *delays* convergence.

Manual inspection. I coded 20 mid-loop failures blind — the sampler shuffles rows and withholds arm labels in a separate key file, because knowing a failure came from an ablated arm would make it trivial to code it as oracle dependence. C6 was therefore not coded but *derived* after unblinding.

arm	C1 detail loss	C2 operation drift	C4 signature	C5 extraction	C7 plus-only	n
A0 `assert`	1	0	1	1	1	4
A1 `masked_assert`	4	3	1	0	0	8
A2 `signature`	4	4	0	0	0	8

The modes differ sharply. Baseline failures are mechanical and idiosyncratic: one nested-code-fence truncation, one `(m, n)` parameter inversion, one extended-test edge case, one garbled recursion. Ablated failures are **almost entirely specification degradation** (8/8 and 8/8 in C1+C2). **11 of the 20 sampled failures are oracle-dependent** — the same task sustained strictly longer under A0 — and 8 of those went from surviving all ten cycles to failing at cycle 1.

Reading the cases, the example output was silently doing *disambiguation*, not merely providing a crutch to copy: it resolved element ordering (Mbpp/462), the return container (set vs list, Mbpp/473), rotation direction (Mbpp/586), whether "pairs of tuples" means pairs *of* tuples or the pair *within* one (Mbpp/120, Mbpp/409), and the treatment of zero digits (Mbpp/745). Notably this model **never echoes the assert (0.0% of cycles)**, so the leak cannot be operating as a copyable answer — which refutes my pre-registered H5 and replaces it with a better account.

A loop the extension broke. Mbpp/226: *"remove the characters which have odd index values"*, anchored `assert odd_values_string('abcdef') == 'ace'`. Under A0 it survives all ten cycles. The model's summary becomes *"return a string containing only the characters at odd-indexed positions"* — a coherent but **inverted** task. Under A0 the expected output `'ace'` contradicts that reading and the model implements the original anyway. Under both ablated arms it fails at cycle 1, emitting `return string[1::2]` → `'bdf'`. The description was always ambiguous; the oracle was resolving it.

A loop it did not. Mbpp/3: *"identify non-prime numbers"*, anchored `assert is_not_prime(2) == False`. Ten cycles under all three anchors. Its summary stabilizes at cycle 1 — *"determine if a given integer is not a prime number by checking for divisibility up to its square root"* — and never moves. The natural-language description fully determines the function, so no oracle is needed. **That is the pattern: the anchor props up precisely the under-specified tasks, and ASL does not distinguish them from the well-specified ones.**

5. Limitations and implications

Threats. (i) Two models from one family, so the ranking question is weakly addressed; the within-model paired comparisons, which are the stronger evidence, do not depend on it. (ii) Few failure events — 4 mid-loop failures in the baseline arm means the Wilcoxon rests on ∼10 discordant tasks, and the 20 coded failures are close to the whole population rather than a sample of it. (iii) MBPP is from 2021 and these models sit at 0.747 pass@1, so contamination is plausible and would inflate all arms equally. (iv) The 150-task subset (mitigated: $\rho = 0.937$). (v) A2 changes information content *and* surface form; A1 is the intermediate rung that partly separates them. (vi) Single run per configuration. (vii) One coder, coding their own experiment; blinding mitigates but does not remove this.

Implications. EvaLoop's ASL is a sound idea undermined by two design details. The loop converges to a fixed point by cycle 2–3, so ten cycles mostly re-derive the same strings while i^2 pays compound credit — which is why ASL tracks pass@1 at $\rho = 0.945$. And the anchor that keeps the harness runnable also re-injects ground truth, which disproportionately rescues under-specified tasks. Both are fixable without abandoning the metric: derive the anchor from the model's own output (A2 costs nothing extra and is strictly more faithful to the self-consistency premise), stop at the first repeated state rather than a fixed budget, and report the sustained-cycle histogram rather than a single i^2-weighted scalar. More broadly, a self-consistency benchmark should account for every channel through which the benchmark's own

answers can re-enter the loop; here one line of scaffolding, present for a legitimate engineering reason, was quietly supplying the disambiguation that the measurement was supposed to be testing.